MongoDB JSON Schema Validation Tutorial

This tutorial explains how to use **MongoDB JSON Schema Validation** to enforce data consistency and ensure that only valid data is inserted into a collection. The tutorial covers:

- Why schema validation is needed
- How to apply validation to new collections
- How to apply validation to existing collections
- BSON types, constraints, and validation rules
- Practical command examples in the Mongo shell

---

1. Need for JSON Schema Validation

In MongoDB, documents are schema-less, meaning you can insert any type of value into fields. For example:

- A field meant for numbers (e.g., `age`) may mistakenly receive a string.
- A `date` field may receive numeric or string values.
- Without restrictions, invalid data is saved without errors.

To prevent such issues, MongoDB provides **JSON Schema Validation**, allowing you to:

- Define field data types
- Mark fields as required
- Set numeric min/max limits
- Allow only particular values (enum)
- Validate embedded documents
- Validate array item types

---

2. Creating a Collection with Schema Validation

MongoDB uses the same `createCollection()` command as usual but with a **second parameter** containing validation rules.

General Syntax

```
db.createCollection("collectionName", {
  validator: {
    $jsonSchema: {
      title: "Schema Title",
      required: [...],
      properties: {
        fieldName: { ...validation rules... }
      }
    }
```

```
  }
})
```

3. Common JSON Schema Fields

3.1 `required`

Specifies fields that must be present in the document.

```
required: ["name", "age", "course"]
```

3.2 `properties`

Defines validation rules for each field.

3.3 BSON Types Allowed

Type	BSON Type Value
string	"string"
integer	"int"
double	"double"
boolean	"bool"
array	"array"
object	"object"
date	"date"

Important shortcuts:

- `int` instead of `integer`
- `bool` instead of `boolean`

3.4 Numeric Constraints

Available for `int` and `double`:

```
minimum: 5,
maximum: 20
```

3.5 Enum

Restrict fields to predefined values:

```
enum: ["BCA", "BTECH", "BSC"]
```

3.6 Validating Array Items

```
bsonType: "array",
items: {
  bsonType: "string"
}
```

3.7 Nested Object Validation

Example structure:

```
bsonType: "object",
required: ["street", "city", "zip"],
properties: {
  street: { bsonType: "string" },
  city: { bsonType: "string" },
  zip: { bsonType: "int" }
}
```

---

4. Complete Example: Creating a New Validated Collection

4.1 Create `students` collection with validation

```
db.createCollection("students", {
  validator: {
    $jsonSchema: {
      title: "Student Object Validation",
      required: ["name", "age", "course"],
      properties: {
        name: {
          bsonType: "string",
          description: "Name must be a string and is required"
        },
        age: {
          bsonType: "int",
          minimum: 5,
          maximum: 20,
```

```
            description: "Age must be an integer between 5 and 20"
          },
          course: {
            bsonType: "string",
            enum: ["BCA", "BTECH", "BSC"],
            description: "Course must be one of BCA, BTECH, BSC"
          }
        }
      }
    }
})
```

---

5. Testing the Validation

5.1 Valid Insert (Accepted)

```
db.students.insertOne({
  name: "Rahul",
  age: 18,
  course: "BTECH"
})
```

5.2 Invalid Insert Examples

Invalid name (number instead of string)

```
db.students.insertOne({
  name: 34,
  age: 18,
  course: "BCA"
})
```

Result: **Error:** `name must be string`

Invalid course (not allowed in enum)

```
db.students.insertOne({
  name: "Rohan",
  age: 19,
  course: "BCOM"
})
```

Result: **Error:** `Course must be one of BCA, BTECH, BSC`

Age out of range

```
db.students.insertOne({
  name: "Amit",
  age: 25,
  course: "BSC"
})
```

Result: **Error:** `Age must be between 5 and 20`

---

6. Applying Validation to an Existing Collection

Use the `runCommand()` function when updating validation rules for a collection that already exists.

General Syntax

```
db.runCommand({
  collMod: "collectionName",
  validator: {
    $jsonSchema: { ...schema here... }
  }
})
```

---

7. Example: Adding Validation to an Existing Collection

Consider an existing collection `personal` with fields:

- name
- age
- married
- dob
- kids
- hobbies
- address (object)

Validation Command

```
db.runCommand({
  collMod: "personal",
  validator: {
```

```
    $jsonSchema: {
      required: ["name", "age", "married", "dob", "kids", "address"],
      properties: {
        name: {
          bsonType: "string",
          description: "Name must be string and required"
        },
        age: {
          bsonType: "int",
          minimum: 1,
          maximum: 100,
          description: "Age must be integer"
        },
        married: {
          bsonType: "bool",
          description: "Married must be boolean"
        },
        dob: {
          bsonType: "date",
          description: "Date of Birth must be a date"
        },
        kids: {
          bsonType: "int",
          description: "Kids must be integer"
        },
        hobbies: {
          bsonType: "array",
          items: { bsonType: "string" },
          description: "Hobbies must be an array of strings"
        },
        address: {
          bsonType: "object",
          required: ["street", "city", "zip"],
          properties: {
            street: { bsonType: "string" },
            city: { bsonType: "string" },
            zip: { bsonType: "int" }
          }
        }
      }
    }
  }
})
```

8. Testing Invalid Inserts on the Existing Collection

Example invalid data:

```
db.personal.insertOne({
  name: 123,
  age: 30,
  married: "yes",
  dob: new Date(),
  kids: 1,
  hobbies: ["cricket"],
  address: { street: "MG Road", city: "Delhi", zip: 110011 }
})
```

Result:

- Name must be string
- Married must be boolean

MongoDB blocks the insert and returns a detailed error message.

---

9. Summary

MongoDB JSON Schema Validation allows you to:

- Enforce strong data typing
- Prevent malformed data
- Restrict allowed values
- Validate arrays and nested objects
- Apply rules to new or existing collections

Key commands learned:

Creating new validated collection

```
db.createCollection()
```

Modifying validation for existing collection

```
db.runCommand({ collMod: ... })
```

Schema validation is a powerful feature that helps maintain data integrity and prevents inconsistencies in MongoDB databases.
